The Independent Courier

VOL. 1, NO. 018

2026-03-30

FEATURES

The Dangers of “Vibe Reporting” About AI

Study Hacks · Study Hacks (Cal Newport)

Last week, news broke that Amazon would be laying off 16,000 workers. Here was the headline from an article about this news published in Quartz:

The implication of this framing is clear: AI is taking jobs.

Nothing in the body of this article contradicts this idea. It describes the number of people laid off and the benefits they’ll receive. It quotes executives who won’t deny the possibility of future job losses. It mentions how Amazon is known for its “cutthroat” corporate culture.

You walk away feeling that the impact of AI on our economy is already getting out of hand.

The only problem is that this reporting omits almost all relevant details.

For a more realistic take, let’s turn toward the financial press. CNBC published an article about these same layoffs featuring a more informative headline:

The article goes on to correctly attribute the layoffs to Amazon’s desire to trim layers of management bureaucracy that built up during the pandemic-era tech hiring boom: “CEO Andy Jassy has looked to slim down Amazon’s workforce after the company went on a hiring spree during the Covid-19 pandemic.”

What role does AI play in all of this? Like many leading companies in the technology sector, Amazon is investing heavily in building its own AI products. Presumably, money is being saved by firing managers, which frees up more revenue to invest in this area. But that’s really it. As the CNBC article elaborates:

“In a blog post, the company wrote that the layoffs were part of an ongoing effort to ‘strengthen our organization by reducing layers, increasing ownership, and removing bureaucracy.’ That coincides with a push to invest heavily in artificial intelligence.” [emphasis mine]

The CNBC article then reports that these massive layoffs actually began for Amazon in 2022 and 2023, following the pandemic, but before ChatGPT was released and the subsequent generative AI revolution began.

Both of these articles cover the same announcement, but they produce two very different impressions. The Quartz article strongly implies that Amazon is firing people because it can now offload their work to AI. (I mean: look at the Andy Jassey quote they included in the sub-head, they clearly wanted readers to believe AI caused these job losses.)

The CNBC article, by contrast, makes it clear that the connection between AI and these layoffs is more coincident than causal.

In recent years, I’ve seen more articles follow the general approach demonstrated by the Quartz example. They identify an alarming,attention-catching fear about AI that seems prevalent in the cultural zeitgeist, and then shape a story to feed the narrative. The key to this vibe reporting strategy is that the articles never make explicit claims. They instead combine cunning omissions and loosely related quotes to make strong implications.

The Quartz article, for example, never concretely states that the 16,000 workers are being replaced with AI; rather, it conveniently avoids mentioning any of the publicly available details about the layoffs that would contradict that idea, and then interleaves quotes about AI’s disruptive potential into the reporting in a highly suggestive manner.

The goal of this type of article is to create a pre-ordained vibe, not to get to the bottom of what’s really happening.

I’m not pointing out this phenomenon to dismiss concerns about AI, but instead because I think this strategy is an obstacle to real action. This type of disingenuous reporting is not going to help us identify the actual problems that require actual solutions. It instead creates a nihilistic sense of inevitable disruption that might drive social media shares, but also numbs people and prevents meaningful responses.

Remember: Nothing about these tools is inevitable, and their impact is far from preordained. We don’t need vibes right now. Reality is too important.

The post The Dangers of “Vibe Reporting” About AI appeared first on Cal Newport .

Making of a Poem: Joyelle McSweeney on “My Fortune”

Joyelle McSweeney · The Paris Review Blog

Fritz Geller-Grimm, CC BY-SA 2.5 , via Wikimedia Commons

For our series Making of a Poem, we’re asking poets and translators to dissect the poems they’ve contributed to our pages. Joyelle McSweeney’s “ My Fortune ” appears in our new Spring issue, no. 255.

How did this poem start for you?

For about a year I found the news so bleak that I turned away from the present tense and made myself a connoisseur of Fortune—the grave goods packed into the Pharaoh’s tomb—his mask, his cats, his casket. From the window of my phone, from the cold black cell of my wakefulness, I would watch rival Egyptologists make competing cases, revolving algorithmically, in and out of view. I watched Cocktails with a Curator , a series of hypererudite videos recorded by Frick Gallery staff from their apartments in New York at the height of lockdown, replayed now in sequence like a journal of the plague year—this swain, his lover, this horse, his Polish rider, this hat, this collar, this pearl. This vial. This tipsy lethal cup.

One night, prowling among my treasures in the dark like a crone-ghost or crow, I saw a glittering promotion for some past Sotheby’s or Christie’s auction of a priceless silver service from the eighteenth century. It was laid out on a dark dining table, where you would expect to see such things in use, yet the pieces were crammed on all together, at once, as you never would expect to see them—all the tureens, all the platters, all the chargers, all the salts. And they were thickly lit, from every angle, as you would also never see in life. The light rinding the silver was unnatural, strange, dead. Some lord had lost his fortune.

When I lost My Fortune, mo stóirín , my little treasure, I felt my whole body twist as in a snare or cinch as I entered my next life—not a life I wanted, not a life anyone would want. Blinking like a stuck drain at all the dead treasure laid out on the table, I thought about figures from Ovid’s Metamorphoses , how they turn at the torso as their new life begins, a life they couldn’t have thought of until they turned into the thought of it. A laurel. A reed. An echo. I wanted a poem that would turn like that, with snares and cinches, that would elapse very fast. I wanted the poem to drain through your fingers like sand.

This poem begins with a photograph of our daughter in the NICU , odi et amo . I love and hate it. Because that’s her face—her tubes, her mask, her tape. The poem drains through the shapes of the dead silver service, then through the figure of Venus’s ankle to a beach where fortune advances and retreats.

What was a particular formal challenge of this poem?

One challenge was to make the poem run out as quickly as possible, as when the tide runs away from the beach, leaves its litter, then rushes back to wipe out even that. The little periwinkles (Littorina littorea , that little litter, my treasure) catch last light then are knocked away. Maybe something else will rise. The stars will rise. The seas.

The most challenging thing about the poem was keeping the opening section this unguarded, keeping the unlovely repetition of the word thinking , as if to suggest a certain horror and immobility in the face of grave loss. Thought from every angle. The unnatural rind of thought.

Joyelle McSweeney is the author of Death Styles and the verse play Dead Youth, or, The Leaks, among other books. She teaches at the University of Notre Dame.

how the creative impulse of utopia improves the world – maria arana zubiate

Maria Arana Zubiate · designboom Architecture

Two natures, one ambiguity, and the danger of its absence

Over time, utopias, often disparaged, have shown their capacity to guide and shape the evolution of the world toward more desirable scenarios. The power of utopia is such that many of today's widely accepted urban proposals— such as collective housing, garden cities, or mass public transport — were once dismissed as utopian. Over the past century, utopia became a powerful tool for accelerating change. After the First World War, the historian, philosopher, and urbanist Lewis Mumford, author of the book The Story of Utopias, argued that the most important task of the moment was to 'build castles in the air,' advocating for a proactive, creative, and visionary attitude toward a world that could not be accepted as desirable or just.

As Mumford pointed out, the history of utopias is, in fact, the history of the world. They emerge as a rejection of the social and cultural context of their time, not as a naive refuge, but as a critical gaze that reveals the shortcomings of their present. Utopia is, in itself, a way of understanding reality, a way of seeing the world that allows us to imagine other rules, other connections, and other architectures. And it begins with an impulse that precedes any method: a creative reaction that pushes us to imagine the impossible.

not all utopias are the same

Lewis Mumford divided utopias into two main categories: escapist utopias, which flee from reality, and regenerative utopias, which seek to transform it. Furthermore, he warned of the dangerous proximity between dystopia and the realized utopia: when the ideal is realized, it runs the risk of degenerating into its opposite. How can we differentiate these utopias and find the boundary that turns them into dystopia? How can we work with this ambiguity to recover the drive that fosters utopian thinking within a dystopia? Here is a small selection of projects from Eutopias, Ou-topias , the main exhibition of the Basque International Architecture Biennial, Mugak, which may help explain this complexity.

Utopias of Escape: Imaginaries Between Critique and Desire

Utopias of escape offer an imaginary refuge from the contradictions of the present and function as cultural safety valves that express the desire for a different possible life and constitute forms of critical escape. From nomadism, voluntary confinement, or adaptive architectures, all of them explore escape as voluntary exclusion—geographical and existential—through the creation of new imaginaries.

New Babylon is the name that Dutch artist Constant Nieuwenhuys gave to a large-scale project developed between 1956 and 1974. Conceived as a global city, free of borders, composed of large, elevated, and transformable structures, New Babylon presents itself as a space in constant metamorphosis, designed to welcome a humanity liberated from productive labor and fully dedicated to play, creation, and experimental living. A territory of freedom and exploration in which architecture ceases to be a fixed framework and becomes a malleable instrument of social experimentation.

Today, Constant's proposal remains remarkably relevant. In the face of a globalized urban model marked by the standardization of spaces, the commodification of leisure, and the centrality of productivity, New Babylon challenges the contemporary imagination by positioning play, mobility, and creativity as the axes of spatial organization. The growing dematerialization of work, the nomadic mobility of broad social sectors, and the expansion of global communication networks seem to give substance—albeit in a fragmentary and contradictory manner—to some of Constant's intuitions. At the same time, the climate and migration crisis, inequality, and the transformation of dwelling practices reopen the interest in thinking of the city as a collective project oriented toward emancipation rather than solely toward economic efficiency.

> *That intermediate space, where the ideal merges with the precarious and the collective with the provisional, reveals the fragility of our ways of living.*
>
> Maria Arana Zubiate

Regenerative utopias: Experimental cartographies for the future

In an urban world marked by deep inequalities, architectural imagination can be a political and transformative tool. Regenerative utopias seek to reverse the physical and social deterioration of cities, not only by recovering what has been lost, but by proposing new ways of living and coexisting. Here architecture ceases to be mere construction and becomes a medium that creates bonds, projects futures, and draws new maps of possibilities. Two experiences stand out in this context:

The Available City , conceived by architect and urban planner David Brown, is a proposal for urban intervention based on a striking statistic: the city of Chicago has approximately 13,000 vacant lots, an area equivalent to twice the size of its downtown core. David Brown proposes viewing these vacant sites as an interconnected system rather than isolated lots, to create new public spaces and reconfigure the urban fabric starting from the plot, its smallest unit. In 2021, this concept was chosen as the central theme of the 4th Chicago Architecture Biennial, becoming a laboratory for urban experimentation. The results included play spaces, sports facilities, temporary cultural centers, and community gardens, demonstrating the versatility of the model and its capacity to generate networks of collaboration between architects, residents, collectives, and public administration.

What can a software developer do about climate change?

Itamar Turner-Trauring

Pines and firs are dying across the Pacific Northwest, fires rage across the Amazon, it's the hottest it's ever been in Paris—climate change is impacting the whole planet, and things are not getting any better. You want to do something about climate change, but you're not sure what.

If you do some research you might encounter an essay by Bret Victor— What can a technologist do about climate change? There's a whole pile of good ideas in there, and it's worth reading, but the short version is that you can use technology to "create options for policy-makers."

Thing is, policy-makers aren't doing very much.

So this essay isn't about technology, because technology isn't the bottleneck right now, it's about policy and politics what you can do about it. It's still written for software developers, because that's who I write for, but also because software developers often have access to two critical catalysts for political change. And it's written for software developers in the US, because that's where I live, and because the US is a big part of the problem.

But before I go into what you can do, let me tell you the story of a small success I happened to be involved in, a small step towards a better future.

Infrastructure and the status quo

About a year ago I spent some of my mornings handing out pamphlets to bicycle riders. I looked like an idiot: in order to show I was one of them I wore my bike helmet, which is weirdly shaped and the color of fluorescent yellow snot.

After finding an intersection with plenty of bicycle riders and a long red light that forces them to stop, I would do the following:

When the light turns red, step into the street and hand out the pamphlet.

Keep an eye out for the light changing to green so that I didn't get run over by moving cars.

Twiddle my thumbs waiting for the next light cycle.

It was boring, and not very glamorous.

I was one of just many volunteers, and besides gathering signatures we also held rallies, had conversations with city councilors and staff, wrote emails, talked at city council meetings—it was a process. The total effort took a couple of years (and I only joined in towards the end)—but in the end we succeeded.

We succeeded in having the council pass a short ordinance, a city-level law in the city of Cambridge, Massachusetts. The ordinance states that whenever a road that was supposed to have protected bike lanes (per the city's Bike Plan) was rebuilt from scratch, it would have those lanes built by default.

Now, clearly this ordinance isn't going to solve climate change. In fact, nothing Cambridge does as a city will solve climate change, because there's only so much impact 100,000 people can have on greenhouse gas emissions.

But while in some ways this ordinance was a tiny victory in a massive war, if we take a step back it's actually more important than it seems. In particular, this ordinance has three effects:

Locally, safer bike infrastructure means more bicycle riders, and fewer car drivers. That reduces emissions—a little.

Over time, more bicycle riders can kick off a positive feedback cycle, reducing emissions even more.

Most significantly, local initiatives spread to other cities—kicking off these three effects in those other cities.

Let's examine these effects one by one.

Effect #1: Fewer cars, less emissions

About 43% of the greenhouse gas emissions in Massachusetts are due to transportation; for the US overall it's 29% (ref). And that means cars.

The reason people in the US mostly drive cars is because all the transportation infrastructure is built for cars. No bike lanes, infrequent, slow and non-existent buses, no trains... Even in cities, where other means of transportation are feasible, the whole built infrastructure sends the very strong message that cars are the only reasonable way to get around.

If we focus on bicycles, our example at hand, the problem is that riding a bicycle can be dangerous—mostly because of all those cars! But if you get rid of the danger and build good infrastructure—dedicated protected bike lanes that separate bicycle riders from those dangerous cars—then bicycle use goes up.

Consider what Copenhagen achieved between 2008 and 2017 (ref):

2008 2018

# of seriously injured cyclists 121 81

% who residents who feel secure cycling 51 77

% who cycle to work/school 37 49

With safer infrastructure for bicycles, perception of safety goes up, and people bike more and drive less. Similarly, if you have frequent, fast, and reliable buses and trains, people drive less. And that means less carbon emissions.

In Copenhagen the number of kilometers driven by cars

Your dev environment matters less than you think

Itamar Turner-Trauring

How do you setup your dev environment? Depending on your language there are many choices of editor, package manager, build tool, linter, on and on. And every article you find will have a different combination of suggested tools, each of which claiming that their list is The Right Way To Do Things.

So which do you choose?

The short answer: it doesn't matter. Your choice of dev environment is meaningless.

The slightly less flippant answer is that, yes, there are some contraints on which tools you should pick, but otherwise you should just pick something and move on.

Let's see why dev environments don't matter that much in the end, and what limited constraints you should apply when choosing your tools.

Learning how to cook

Imagine you're training to become a chef. You will need to learn how to use a knife correctly, to chop and dice safely and quickly.

And yes, you need a sharp knife. But when you're starting out, it doesn't matter which knife you use: just pick something sharp and good enough, and move on. After all, the knife is just a tool.

The people eating the food you cook don't care about which knife you used: they care how the food tastes and looks.

After six months in the kitchen, you'll start understanding how you personally use a knife, what cuisines you want to pursue, what techniques you want to vary. And then you'll have the knowledge to pick a specific knife or knives exactly suited to your needs.

But remember: the people eating your food still won't care which knife you used.

Choosing a dev environment

When you use a website, you don't care which build tool the programmer used. When you run an app, you don't care which editor they used. You want the software to work, to do what it says, to be easy to use, to get out of your way—and you don't care how they did it.

And that applies just as much to the users of your code: they don't care which tools you used.

And when you're starting out, whether programming in general or a new language or framework, you don't know how you will like to work. So instead of obsessing over finding the ideal development environment and toolchain, just pick tools that are good enough:

Popular: So you can easily find help.

Easy to get going: Your goal is ship useful code, and as a beginner time spent fiddling with your dev environment won't help with that.

Once you have enough experience, you will start developing opinions. You might become choosy about which tools you use, or end up customizing them to your needs. You might even write an article about your particular dev environment and favorite tools.

But however strong your preferences are, chances are that given tools you don't quite like as much, you will still do just fine. If you know what you're doing, you can chop vegetables with any sharp knife, even if it's not your favorite.

Tired of scrambling to get your job done?

If you were productive enough, you could take the afternoon off, confident you'd produced high value work. Not to mention having an easier time finding a new job when you need one.

Learn the secret skills of productive programmers .

Static initializers will murder your family

Monica Dinculescu

But only if your family is code.

So this is a bit of a terrible blog post because a) it's about a really obscure atrocity that happens in C++ (as opposed to the common atrocities that happen in C++ on the regs) and b) there are not enough funnies in the world to make up for it. I recommend skipping it if you've just eaten, are feeling light-headed, or don't want to make eye contact with C++. As a general policy, you should probably never make eye contact with C++. It can smell fear.

Programmer, meet static initializers

We're going to be talking about static class objects, or objects defined in a global/unnamed namespace, such as these fellas:

namespace { static const std :: string kSquirrel = "sad squirrel" ; static const Superhero batman ; } /\/ or class Foo { static const std :: string panda_ = "also a sad panda" ; }

Static initialization is the dance we do when creating these objects. This is not a dance we do when we initialize things with constant data (like static int x = 42); the compiler sees that the thing after the = is constant and can't change, so it can inline it. However, if you try to initialize a variable by running code (e.g. static int x = foo()), then this is not a constant anymore, and it will result in a static initializer. In C++11, I think constexpr will let you hint to the compiler that the thing after the equal is a constant expression, if it is that, so it can compute it at compile-time. I don't get to use a lot of C++11, so this is still about nightmares of C++ past, and I don't think constexpr will do away with all of the murders anyway. Finally, the compiler promises you to run all the static initializers before the body of main() is executed. That, unfortunately, doesn't mean much.

Why static initializers are bad news bears

As Douglas Adams, the inventor of C++ said, static initializers have "made a lot of people very angry and been widely regarded as a bad move". Apart from being hard to spell, they tend to throw up on your shoes:

Static variables in the same compilation unit (or the same file) will be constructed in the order they are defined. This means that this code is predictable, and always does exactly what you think it does. This is also the last of the good news:

namespace { static Superhero batman ; static Superhero robin = batman . getSidekick (); }

Static variables in different translation units are constructed in an undefined order. This is so terrible it has its own name: the static initialization order fiasco . It goes like this:

/\/ In x.cpp: static Superhero batman ;

/\/ In y.cpp: static Superhero robin (batman . getSidekick ()); /\/ If that wasn't believable, imagine it was something like: /\/ static Superhero robin(BestSuperhero::batman); /\/ where BestSuperhero is a namespace or a static class and /\/ you call batman.getSidekick() in robin's constructor.

Yup. That's it. Whether x.cpp or y.cpp gets compiled first is not defined (because C++), which means if y.cpp gets compiled first, batman hasn't been constructed. You know what happens when you call getSidekick() on an uninitialized object? Regrets happen.

We're not done yet. Why have insanely terrible code when you can have insanely terrible EXPENSIVE code! Evan Martin has a really, really good post about this, but the tl;dr is that because the static initializers need to happen before main() , that code needs to be paged, which leads to disk seeks, which leads to awful startup performance. Seriously, read Evan's post because it's amazing.

Spotting static initializers in the wild: an incomplete manual

Here are some examples of things that are and aren't static initializers, so that at least we know what we're looking for before we try to fix them.

/\/ Both of these are ok, because 0 is a compile time constant, so it can't /\/ change. The const doesn't make a difference; it's the thing after /\/ the = sign that makes the difference. static const int x = 0 ; static int y = 0 ;

/\/ Below, both the pointer and the chars in the string are const, so the /\/ compiler will treat this as a compile-time constant. So this is ok /\/ because both the thing before and after the = sign are constant. static const char panda [] = "happy panda" ;

/\/ This, however, calls a constructor, so it's not ok. static const std :: string sad_panda = "sad panda" ;

static int a = 0 ; /\/ This is not ok, because the thing after the = sign isn't a const, /\/ so it can change before b is initialized. static int b = a ;

/\/ This has to call the Muppet() constructor, and who knows what that /\/ does, so it's definitely not a const, and a case of the static initializers. static Muppet waldorf ;

Them's the breaks

There's a couple of ways in which you can fix this, some better than others:

The best static initializer is no static initializer, so try consting all your things away. This will take you as far as defining an array of strings, for which you can't pray the initializer away. (Trivia: Praying The Const Away™ is what I call a const_cast)

Place all your globals in the same compilation unit (i.e. a massive constants.cpp file). You can certainly try this, but if

37signals Isn't Smarter Than You, But They Are Different

Nate Berkopec

A recent episode of the Rails Business Podcast was about "striving for ideal code". The hosts are both typical "Rails Indie" owner/operators: a small team building a small SaaS.

They were discussing a new podcast which is out from 37signals called "Recordables" . The discussion was envy for how good and clean 37signals' code was. This is a fair assessment. They've been open-sourcing a lot of real world products lately, including:

Campfire , the chat app.

Fizzy , the Kanban tool.

If you read them, the code looks great. We've only seen glimpses of this Rails style, which David has alluded to or shown small parts of in interviews or conference talks. Until now, we couldn't see what the Rails "house style" is because we never had a real open source application written in that style. Now we do.

The discussion on the podcast was mostly a sense of envy of how good the good was, and wondering: "Why doesn't our code look that good?"

Covet not thy neighbor's codebase.

The reason why your code doesn't look "as good" as 37signals' is because your engineering strategy is not the same as theirs .

37signals, for all of its faults , has a unique engineering strategy, which I would summarize as:

Stay small in terms of headcount to revenue ratio.

Ruthlessly cut scope . 37signals' products usually do far, far less than their competition.

Hire the top 10% of engineers.

Most of the businesses I've ever worked for cannot say that any of these three points are honestly part of their strategy.

Every time 37signals puts out a codebase or talks about their size in public, I am always shocked at how few LOC they are.

Basecamp Next/2 was originally 10,000 LOC .

Basecamp 3 was 18,000 LOC when released .

Fizzy is 7,500 non-test LOC.

Campfire is just 2,500 non-test LOC.

That is fucking insane . 37signals employs 25-30 technical employees, meaning that for each individual codebase, they're maintaining under 2,500 lines per person. Most companies manage 10x that per headcount. Most indies ship 100,000+ line Rails apps with 1 to 3 people. I know, because they've been an important part of my consulting client base for the last decade!

The default engineering strategy for most companies is: if we ship one more feature, we will make more money . From that follows if we ship more features, we need more people to ship faster and maintain what exists , and from that follows our ability to hire good people won't keep up with how much we can pay and how fast we need to hire , so they compromise. You end up with the exact opposite engineering strategy to 37signals.

And so, 37signals' strategy creates a flywheel: you have more developers maintaining fewer LOC, which allows them to apply more fixup/review cycles to the same lines over and over, whereas most shops have to just throw shit over the wall and move on to the next thing. That creates a cleaner codebase, which helps reduce needed headcount, which also increases your likelihood of hiring great people (because great people want to work on great code).

Most businesses cannot adopt the 37signals strategy because they are fundamentally unwilling to relinquish the mindset that "more features equals more money". And if 37signals has been consistent in anything when developing software, it's that they will not allow scope creep.

Another reason they can't implement this strategy is they don't have product market fit . If you're default-dead and you're gonna run out of money, or if you're kind of coasting along at $1m a year with 5 people, you need to try lots of new things to attain PMF. They simply don't have the luxury of running at $5 million of ARR per employee and spending all this time polishing everything they've got. They need to throw shit against the wall and study what sticks.

37signals is not full of geniuses. They are not better than you. What they've done is adopted a strategy that lets them ship better code.

And that's the rub: most software businesses don't know any other way to make money other than more . Ask yourself: is that really the only way?

10 Interview Questions Every JavaScript Developer Should Know in 2024

Eric Elliott · Eric Elliot (JavaScript Scene)

The world of JavaScript has evolved significantly, and interview trends have changed a lot over the years. This guide features 10 essential questions that every JavaScript developer should know the answers to in 2024. It covers a range of topics from closures to TDD, equipping you with the knowledge and confidence to tackle modern JavaScript challenges.

As a hiring manager, I use all of these questions in real technical interviews on a regular basis.

When engineers don't know the answers, I don't automatically reject them. Instead, I teach them the concepts and get a sense of how well they listen, and learn, and deal with the stressful situation of not knowing the answer to an interview question.

A good interviewer is looking for people who are eager to learn and advance their understanding and their career. If I'm hiring for a less experienced role, and the candidate fails all of these questions, but demonstrates a good aptitude for learning, they may still land the job!

1. What is a Closure?

A closure gives you access to an outer function's scope from an inner function. When functions are nested, the inner functions have access to the variables declared in the outer function scope, even after the outer function has returned:

const mySecret = createSecret("My secret");
console.log(mySecret.getSecret()); /\/ My secret

mySecret.setSecret("My new secret");
console.log(mySecret.getSecret()); /\/ My new secret

Closure variables are live references to the outer-scoped variable, not a copy. This means that if you change the outer-scoped variable, the change will be reflected in the closure variable, and vice versa, which means that other functions declared in the same outer function will have access to the changes.

Common use cases for closures include:

Data privacy

Currying and partial applications (frequently used to improve function composition, e.g. to parameterize Express middleware or React higher order components)

Sharing data with event handlers and callbacks

Encapsulation is a vital feature of object oriented programming. It allows you to hide the implementation details of a class from the outside world. Closures in JavaScript allow you to declare private variables for objects:

Curried functions and partial applications:

/\/ A partial application is a function that has been applied to some, /\/ but not yet all of its arguments. const increment = add(1); /\/ partial application

increment(2); /\/ 3

2. What is a Pure Function?

Pure functions are important in functional programming. Pure functions are predictable, which makes them easier to understand, debug, and test than impure functions. Pure functions follow two rules:

Deterministic — given the same input, a pure function will always return the same output.

No side-effects — A side effect is any application state change that is observable outside the called function other than its return value.

Examples of Non-deterministic Functions

Non-deterministic functions include functions that rely on:

A random number generator.

A global variable that can change state.

A parameter that can change state.

The current system time.

Examples of Side Effects

Modifying any external variable or object property (e.g., a global variable, or a variable in the parent function scope chain).

Logging to the console.

Writing to the screen, file, or network.

Throwing an error. Instead, the function should return a result indicating the error.

Triggering any external process.

In Redux, all reducers must be pure functions. If they are not, the state of the application will be unpredictable, and features like time-travel debugging will not work. Impurity in reducer functions may also cause bugs that are difficult to track down, including stale React component state.

3. What is Function Composition?

Function composition is the process of combining two or more functions to produce a new function or perform some computation: (f ∘ g)(x) = f(g(x)) (f composed with g of x equals f of g of x).

Polymer 1.x Cheat Sheet

Monica Dinculescu

This is a cheat sheet for the Polymer 1.x library. It helps you write Web Components, which are pretty □□□. If you're interested in the Polymer 2.0 cheat sheet, it's here . If you think something is missing from this page, tell me about it!

Defining an element

Defining a behaviour

Lifecycle methods

Data binding

Properties block

Observing added and removed children

Style modules

Styling with custom properties and mixins

Binding helper elements

Defining an element

Docs: registering an element , behaviours , shared style modules

Polymer ({ is : ' element-name ' , /\/ All of these are optional. Only keep the ones you need. behaviors : [], observers : [], listeners : {}, hostAttributes : {}, properties : {} });

Defining a behaviour

Docs: behaviours .

Defining a behavior to share implementation between different elements:

MyNamespace . MyFancyBehaviorImpl = { /\/ Code that you want common to elements, such /\/ as behaviours, methods, etc. }

MyNamespace . MyFancyBehavior = [MyFancyBehaviorImpl , /* You can add other behaviours here */];

Using the behavior in an element:

Polymer ({ is : ' element-name ' , behaviors : [MyNamespace . MyCustomButtonBehavior] /* ... */ });

Lifecycle methods

Docs: lifecycle callbacks .

Polymer ({ registered : function () {}, created : function () {}, ready : function () {}, attached : function () {}, detached : function () {} });

There's an attributeChanged callback as well, but that's very rarely used.

Data binding

Docs: data binding , attribute binding , binding to array items , computed bindings .

Don't forget: Polymer camel-cases properties, so if in JavaScript you use myProperty , in HTML you would use my-property .

One way binding: when myProperty changes, theirProperty gets updated:

Two way binding: when myProperty changes, theirProperty gets updated, and vice versa:

Attribute binding : when myProperty is true , the element is hidden; when it's false , the element is visible:

Computed binding : binding to the class attribute will recompile styles when myProperty changes:

_computeSomething: function(prop) { return prop ? 'a-class-name' : 'another-class-name'; }

Docs: observers , multi-property observers , observing array mutations .

Adding an observer in the properties block lets you observe changes in the value of a property:

properties : { myProperty : { observer : ' _myPropertyChanged ' } },

/\/ The second argument is optional, and gives you the /\/ previous value of the property, before the update: _myPropertyChanged : function (value , /*oldValue */) { //... }

In the observers block:

observers : [' _doSomething(myProperty) ' , ' _multiPropertyObserver(myProperty, anotherProperty) ' , ' _observerForASubProperty(user.name) ' , /\/ Below, items can be an array or an object:' ' _observerForABunchOfSubPaths(items.*) ']

Docs: event listeners , imperative listeners .

listeners : { ' click ' : ' _onClick ' , ' input ' : ' _onInput ' , ' something-changed ' : ' _onSomethingChanged ' }

Properties block

Docs: declared properties , object/array properties , read-only properties , computed properties .

There are all the possible things you can use in the properties block. Don't just use all of them because you can; some (like reflectToAttribute and notify) can have performance

Week 2

Monica Dinculescu

Still into soups: I made a bomb cream of broccoli. I also bought a new vegan broth base to fuck with because buying the cartons of broth is like buying bottled water aka: bad and wasteful.

My dog's skin is turning grey. I know this sounds funny but it's like, a thing. There could be a number of reasons for this, but the most likely one is that she's an inbred dumbass and allergic to something in her food, like chicken. Or she could be dying. In any case, she's now on Fancy Food™ which makes her insufferable. Related: pet nutrition is absolutely bonkers.

If you're trying to gauge what level of pandemic mental breakdown I'm at, I dyed my hair pastel pink in the bathroom last week (Manic Panic blessing my life since 2001), and this week I "rescued" a very sad looking croton from the grocery shop. I got emotional that it was ugly and unloved and nobody else was going to care for it because people only buy pretty plants, so I HAD to adopt it. I paid 3$ for this privilege of bringing a plant with broken leaves into my house.

New season of drag race! I know this will be very polarizing, but I kind of love how insane Utica is. I also love Gottmik (and that Ru has finally pivoted to saying "and may the best drag queen win") and Tamisha. Still don't understand the pork chop gimmick, and I haven't eaten a pork chop in like a year and a half so it makes me hungry.

I had some really good meetings at work with super creative and inspiring people, and it's spilling into my personal life: I have 2 (TWO) ideas for generative art projects, and 0 (ZERO) motivation to actually do them. I also haven't finished the Bach Menuet I started last week. What do I do instead with my evenings you ask? Watch every single Trixie Mattel video on YouTube because that's the journey I'm on.

Week 17

Monica Dinculescu

I've been skipping weeks because a) literally nothing happens and b) I don't have a good system to update these notes. They're a markdown file on a GitHub repo, and I kind of need a computer to edit it, but I also kind of don't open my computer that much these days? I also keep forgetting which day is Monday. Room for improvement.

I wrote a blog post about how I generated some tree rings in JavaScript and then carved them as a linocut. It doesn't actually contain any JavaScript, but it does have a lot of pretty images.

The last normal thing I did before the panini started was go to Japantown and stock up on apocalypse supplies (snackos, mucho ramen, milk tea powder, korean face things). The first normal thing I did with my 1 vaccine shot was go to Japantown and restock all the things. This wasn't on purpose, but I am pleased with the serendipity. It is cherry blossom season, so it was very pretty, HOWEVER, some racist asshole vandalized two of the oldest cherry trees there in January. Literally chopped down all of the branches, one at a time. What the actual fuck.

If you're against plastic (why aren't you?) and use deodorants, Dove has started selling refillable ones. I got mine from Target. The refill itself still comes in plastic, but overall it's far less plastic than the obnoxious amount the normal ones have. I'm excited about this not because this deodorant is particularly amazing, but because Dove is a HUGE brand, and having mainstream brands start looking into more reusable, less-plastic products is a small but exciting progress.

Speaking of waste, Zach bought us me the most amazing thing: a foodcycler !!! We've been composting for years, but I've recently started getting lazier about putting the compost back in the freezer when I'm done with it, so our idiot dog has been stealing a lot of compost (which is full of bad things for her like coffee grounds, onion peels, literally half a spaghetti squash rind she ate and threw up for 8 hours). This foodcycler thing takes the compost and dehydrates it and grinds it and in 4 hours is done and gives you back fertilizer at like a tenth of the original volume. We've had it for a week and it's honestly THE MOST innovation I've seen in my kitchen.

Related: pet nutrition is absolutely bonkers.

Monica Dinculescu

The first normal thing I did with my 1 vaccine shot was go to Japantown and restock all the things.

Monica Dinculescu

Chrome extensions for quick site redesigns

Monica Dinculescu

There's this thing I hate about the modern web which is that sites are rarely one giant html file filled with goodies. You can't just "run a site" locally. You need an npm or a gulp step or a docker if you're lucky. And probably a local server, but not the one you have installed. Which, I mean, makes sense, because modern web sites are big and powerful and have complicated front-ends and do more things than a giant html file would. But it also kind of sucks because the build ceremonial sacrifices can be a bit overwhelming. Maybe you just want to see what the links would look like if they didn't have underlines. Maybe you want to change the fonts. Maybe you're never even going to ship these changes, you just want to get a feel for them.

Boy, have I got an idea for you: Chrome extensions. Hear me out. A Chrome extension is a bit of code that runs on a specific page (or set of pages). It can be anything you want. In particular, it can be a CSS stylesheet.

Then, re-theming a site is just a matter of installing this Chrome extension. If you want to share it with people, you can just zip it up and send it around. It's obviously not production ready, but it's amaaaaazing for prototyping.

I made a glitch project that gets you started with writing a Chrome extension that injects a CSS stylesheet. There's only one thing you need to know: this stylesheet is a User Agent stylesheet, which means it has the lowest specificity. So some of its styles won't get applied unless you slap some !important s on it (or have extra specific selectors). Or, if you have an ID, you can do my favourite CSS hack ever that I learnt from Surma and will take to the grave:

#foo#foo { /* this is a really winner #foo selector */ color : red ; }

That's it! If you want to see an example of such an extension in the wild, I made picasso , which is just a pretty Google Calendar theme. Originally it was just a local extension I kept on my machine, but eventually I published it because I realized other people may want to give their calendar a bubble bath. Anyway, happy retheming!

The Best Way to Keep Track of New Children's Books

Community · Book Riot

Every month, there's a flood of new children's books coming out clamoring for a spot on your TBR. But with so many competing for your attention, it's easy to lose track of the ones you're most excited about. And often, you just end up hearing about the titles with the biggest marketing budgets.

So, how do you keep up with new releases without it spiraling into a part-time job of catalogue reviews and spreadsheets? That's where the New Release Index comes in.

The New Release Index is a database of upcoming books, curated by Book Riot. It's organized by release date, and you can filter by genre: above is a sneak peek of some of the middle grade and picture books out in April.

Here's how it works: scroll through the covers until you find one that catches your eye. Click on the cover for the book description, and then save the titles you're interested in on your Watchlist.

The best part is that the New Release Index is included in your All Access subscription. For $6 a month, you not only get the New Release Index, but also all of Book Riot's paid content.

Sign up for All Access to get started, or you can check out our guide to the New Release Index for more information.

Here's how it works: scroll through the covers until you find one that catches your eye.

Community

ANALYSIS

TED's new chapter begins!

Chris Anderson · TED Blog

Today I'm sharing a momentous announcement. After a nine-month global search, we have found a beautiful answer to the question of who will carry TED forward into the future.

Why this moment matters

When we announced this process back in February, our guiding question was simple: Who can best take on TED's unique blend of offerings and values — not just for the next few years, but for the coming decades? Our advisers, LionTree, heard from nearly 100 thoughtful and passionate groups — spanning individuals, foundations, media organizations, technology platforms, investor groups, and universities. (We even received generous offers to purchase TED outright.) The breadth, ambition, and imagination of what was offered were truly inspiring.

But for me — and for everyone on TED's leadership team — the answers weren't just about capital or scale. They were about stewardship, values, and a shared belief in giving ideas away, trusting community, preserving independence, and amplifying human possibility.

From that landscape of offers, we made a decision grounded in the guardrails we set in the original blog post announcing the search:

Ambition for transformation, not just incrementalism — the next steward must have a truly compelling vision for how TED can scale up its impact.

Protection of all that people hold dear about TED : the powerful experience of our main conference, our free TED Talk videos, our cherished communities of volunteers, and our determined optimism that a better future can be built.

An open tent, diverse voices, global reach, nonpartisanship — those are nonnegotiable.

Editorial and intellectual integrity must be protected against any conflicts of interest or agenda.

TED must never be sold out to commercial interests , either now or down the line.

We seriously considered for-profit joint ventures that could allow TED's rapid growth and infuse TED with significant capital. But the more we thought through the possibilities and reminded ourselves of the extraordinary culture of generosity that infuses this community, the harder it was to picture a scenario where TED's ultimate controllers might one day prioritize profits over mission.

So we made a key decision...

TED's Nonprofit status must endure — TED will stay a nonprofit, independent of external commercial control. We will proudly continue our transparent mission of providing knowledge, insights and inspiration freely to anyone in the world.

And now you will understand why I am so excited at where we ended up....

Meet TED's new Vision Steward... Sal Khan!

Sal Khan and Chris Anderson at TED2023 in Vancouver, BC, Canada. (Photo: Gilberto Tadday / TED)

It is my delight to introduce Sal Khan , founder and CEO of the incredible nonprofit Khan Academy. Sal will join TED's board as TED's new Vision Steward , while continuing his full-time role leading Khan Academy. Sal understands, in his bones, what it means to build a global educational platform rooted in the power of what technology and AI can enable in our connected world. Sal's story is well known: starting with math videos and software to help his cousins, he grew Khan Academy into a trusted, free learning resource for students — and a valued partner for schools and districts — with over 170 million registered users in over 50 languages. He has blended pedagogical rigor, technology (including the powerful use of AI), and ingenuity in service of learners everywhere. He has attracted and retained an extraordinary team. He has partnered with the world's most thoughtful philanthropists and recruited one of the most powerful boards in the nonprofit space. Perhaps most importantly, he's won the respect and gratitude of countless numbers of educators, parents, and students worldwide.

What makes Sal especially compelling for TED is that he's shown he can craft a vision that's both breathtakingly ambitious and achievable in today's technological landscape. And he's done so inside a nonprofit culture. Add to this his lifelong dance with the power of curiosity, his hunger for crosscutting ideas, and his belief that bridging divides is not a side project, but a mission. Well, you can see why I'm excited.

Khan Academy and TED will continue to operate independently of each other, but Sal has already shared with me electrifying sketches of how TED could find new ways of empowering humanity by tackling lifelong learning. He also sees how TED could play a meaningful role in helping address the world's growing division and social isolation. We're not announcing any details today. But watch this space... It's about to get seriously exciting.

While Sal will serve as a guiding steward for vision, the day-to-day execution will fall to a new CEO. For the past 12 years Jay Herratti has given sterling service to TED, initially as head of TEDx, and for the past five years as CEO, where he heroically turned my often wild ideas into effective operational reality. During that time he brilliantly dealt with the challenges of a pandemic and a brutally unstable media landscape. Jay has already given many more years to TED than he initially intended, and is now ready for his next chapter in which he intends to focus on board-level work.

The secret skills of productive programmers

Itamar Turner-Trauring

This article was written during abnormal circumstances, with much of the planet under lockdown due to the COVID-19 pandemic. Parents with children at home have far less time, and pretty much everyone is feeling stressed and distracted.

Under more normal circumstances there are only so many hours in the day to do your job; now it's even worse. And yet work needs to get done: code needs to get written, features need to be shipped, bugs need to be fixed.

Faced with an ever growing list of tasks, how do you get everything done?

The short answer is, you can't. You will never get everything done.

What you can do, though, is choose the right work, the most valuable work, the most useful work, the work with most leverage. Choose the right work and you can gets orders of magnitude improvement in your output.

Let's see how.

The goal: increased output

Your output as a programmer is based both on your productivity and on how much time you work:

Output = Productivity × Time Worked

The first thing to notice is that there is a hard limit on how much increasing your working hours can help. After all, there are only 168 hours in a week.

If you never slept, ate, or did anything but work—and this will literally kill you—you can work 4.2× as much as a 40-hour workweek, and that's it. And even with smaller increases in work hours, the gains quickly decline. As you work more hours you'll become fatigued and make more mistakes; beyond a certain point those extra work hours will decrease your productivity, canceling out any gains.

What is output for a programmer?

Since increasing working hours isn't really an option, the key to increasing your output is increasing your productivity. Productivity is the output you produce in each fixed unit of time, for example:

Productivity = Output per week

If you're going to improve your productivity, you need to understand how to measure output.

The obvious measure is how much code you write: the more code, the better. This measure is obvious, popular, and completely wrong.

All other things being equal, is it better to implement the same feature with 10 lines of code, or 10,000 lines of code? If we measure output by code produced, the latter solution is better, but in most cases a 10 line solution is preferable to 10,000 lines. More code means higher maintenance costs, not to mention more opportunities for defects.

Your job as a programmer is not writing code, your job is solving problems : software is a tool, a means to an end. Software becomes valuable because of the problems it solves.

As a rough measure, your output as a programmer can be measured by the problems you solve: the more significant the problems you can solve, the better.

If you work for a business, significance eventually translates directly or indirectly into monetary terms: money made or money saved. In other areas you can come up with domain-specific concrete measures of usefulness: number of people served, carbon emissions reduced, number of scientists using your software, and so on.

Note: If the problems you solve produce negative value you will become anti-productive: the better you are at your job, the more damage you will cause.

If making money hurts people or the environment, your work may be productive for your employer but anti-productive for society as a whole. So make sure you're carefully considering the ethical consequences of your actions as a worker.

How to increase productivity

Given the above, here's how you can increase your productivity:

Find the most significant problem you can work on.

Come up with the most efficient solution to that problem.

Implement the solution with minimum wasted time.

Let's go through these steps one by one, and see why they're key to productivity.

1. Find the most significant problem

Let's consider our formula for productivity again:

Productivity = Significant problems solved / Week

There are many problems you could be working on, so first you have to choose one. If you could solve either of these problems, should you be working on:

Implementing a particular missing feature; this will increase revenue by $50,000.

Fixing a bug that was decreasing customer retention; this will increase revenue by $1,000,000.

Kafka's Misdiagnosis

Aaron Schuster · The Paris Review Blog

In a diary entry from February 1922, Franz Kafka writes of a deal he made with madness:

There is a certain failing, a lack in me, that is clear and distinct enough but difficult to describe: it is a compound of timidity, reserve, talkativeness, and half-heartedness; by this I intend to characterize something specific, a group of failings that under a certain aspect constitute one single clearly defined failing (which has nothing to do with such grave vices as mendacity, vanity, etc.). This failing keeps me from going mad, but also from making any headway. Because it keeps me from going mad, I cultivate it; out of fear of madness I sacrifice whatever headway I might make and shall certainly be the loser in the bargain, for no bargains are possible at this level.

The Kafkian protagonist (including the "I" of Kafka's letters and diaries) is a loser who cannot make "any headway," a schlemiel who secretly cultivates failure as the means of his persistence. The subject must lose, must fail; that's the deal made with madness. Conversely, does this not imply that a successful Kafka would be not a socially well-adjusted, non-neurotic, even happily married Kafka, but rather a mad Kafka, one forced to pay a high price for not sacrificing headway in his pursuit, for going all the way to the end of his investigations? In "Investigations of a Dog," the philosopher dog speaks of wanting to feed on the bone marrow of all the dogs, the marrow of truth—but then turns around and avows that this marrow is "no food; on the contrary, it is a poison." Similarly, what if Kafka nourished himself on failure to avoid being poisoned by the truth he was seeking?

There is something profoundly unhinged about the Kafkian universe. In the first book-length study of Kafka in English (a rather eccentric work, largely forgotten today), Paul Goodman put it sharply: Relax your vigilance and "the entire order of the world will fly in pieces." Kafka himself once called waking up "the riskiest moment": "If you can manage to get through it without being dragged out of place, you can relax for the rest of the day." It's as if the interval between sleep and waking were not only a matter of fuzzy consciousness but also an ontological blurriness, threatening to open a rupture in the fabric of space-time where all sorts of demons might appear, like agents coming to arrest you for an unknown—and unknowable—crime, or a giant insect substituting for your formerly human self. Schizo- in Greek means cleft or split, and apart from the moment of awakening, there are many such figures of schizoid rupture in Kafka's universe. "A Little Woman" opens with a delirious detail: "I have never seen a hand with the separate fingers so sharply differentiated from each other as hers; and yet her hand has no anatomical peculiarities, it is an entirely normal hand." The too-finely-spaced fingers signal a subtle breach in the order of things, a breach into which the narrator can't help but plunge.

It is true that one of the recurrent motifs or atmospheres in Kafka is a kind of nonchalant absurdity or normal insanity, which gives to his writing much of its dry humor. Like when Gregor Samsa turns into a bug and no one's really shocked, or when Blumfeld comes home to his apartment to discover a pair of magical bouncing balls, which he finds bothersome but not particularly extraordinary. "Blumfeld, an Elderly Bachelor" would appear to illustrate the schizophrenia diagnosis, with its quasihallucinatory pair of celluloid balls that keep jumping up and down, doggedly following an increasingly exasperated Blumfeld. But within this delirium lies a fundamentally neurotic problem: on the one hand, the lonely bachelor is frustrated and cannot fulfill his desire (to find a life companion); on the other, a strange enjoyment keeps popping up where it's least expected (the unwanted "companions" hopping around him). Such is the paradox of a bachelor's existence: Loneliness can never be dispelled, and solitude is always interrupted by an intruder. Loneliness is incurable, yet one is never left alone. Likewise, the dog in "Investigations of a Dog" recounts how he was first launched on his philosophical quest by a psychedelic musical concert he stumbled upon in his youth. However, it's not the intensities of light, movement, and sound or the violently reality-bursting spectacle (its "deterritorializing" force, in Deleuze and Guattari's language) that grip the dog; rather, it's the silence of the musicians, their refusal to answer his questions. This silence triggers something in him, destining the rest of his life to repeat this primal scene. His adult quest is a philosophical neurosis, organized around the posing of questions and the nonreception of answers.

Another example: the digging animal in "The Burrow," with his constant fear of predators and obsession with defense, might easily be taken for a paranoid psychotic. But rather than being possessed by the certainty of persecution, he is riven by doubts, admitting that he doesn't know what the enemy knows or if he's plotting against him; near the end of the story he even claims, "I have reached the stage where I no longer wish to have certainty." As the psychoanalyst Darian Leader has argued, if there's one thing that separates neurosis from psychosis, it's certainty. What "The Burrow" brilliantly illustrates is the warped neurotic logic by which one clings more to one's defenses than to the life they are supposed to be defending. Indeed, many of Kafka's abiding themes point to neurosis rather than schizophrenia: the ambivalent relation to authority and the ever-frustrated desire for official permission and status; delay, deferral, postponement, and procrastination; compulsive overthinking (Kafka makes virtuosic use of the word but —the Belgian Germanist Herman Uyttersprot once dubbed him the Aber Mann); misunderstanding and the equivocations of interpretation; a floating sense of guilt, whose cause is unknown; the tortuous intricacies of grievance and complaint; and above all, failure—the failure to reach one's goal or simply to make it from point A to B.

Samuel Beckett held a similar view. In a 1956 interview, Beckett underlined a certain serenity in Kafka's writing: "The Kafka hero has a coherence of purpose. He's lost but he's not spiritually precarious, he's not falling to bits." He

Sunday at La Bombonera

Juan Villoro · The Paris Review Blog

Clásicos are like Christmas for football. In these high-tension matches between fierce rivals, expectation almost always outstrips results. For months, fans visualize goals with the unrealistic yearning of a child who hopes for a new PlayStation from Santa Claus in exchange for a few cookies left out for his tired reindeer.

For me, the Superclásico between Buenos Aires's Boca Juniors and River Plate on May 4, 2008, was preceded by thirty-four years of anticipation. In 1974 I went to the Estadio Monumental to see River–Boca, but I had never been to the reverse fixture in La Bombonera, that exceptional stadium that should have been examined by Elias Canetti in Crowds and Power.

The wait had charged the occasion with so much emotion that it was almost a shame it actually had to take place. Friends from Mexico, Colombia, and Spain had all similarly circled the date of May 4—the Argentine derby appeals not only to those who sleep in shirts emblazoned with the Quilmes beer logo but to an entire global tribe.

Like Everest or the Mona Lisa , the fame of Boca's stadium is impossible to deny—look no further than the crowds of tourists who come to snap pictures. But does it really represent the pinnacle of footballing passion?

I spoke about the Superclásico with the taxi driver who picked me up at the Ezeiza airport and he replied with indignation: "But we hate each other more!" He came from Rosario and was referring to the bad blood between his hometown clubs Newell's Old Boys ("the Lepers") and Rosario Central ("the Swine"). On the drive he told me about his family's marvelous wrath and the betrayal of his aunt Teresita, the heretic who refused to support the Swine. At the core of his story was the issue of rancor: on its biggest days, football comes down to contempt, and nobody hates each other more than Swine hate Lepers. In his opinion, the lesser rivalry between Boca–River was inflated by the press. The driver summed up his argument with theological flair: "God is everywhere, but performs his tricks in Buenos Aires."

Don't Worry: It's Just the Earth Shaking

On April 16, my friend Daniel Samper Pizano, a renowned Colombian journalist, organized a dinner in Madrid to prepare for the Superclásico. The final of the Copa del Rey had just concluded between Valencia and Getafe, but we weren't interested. We preferred to talk about the football of the future, which was to say the upcoming game on May 4. The other guest justified our interest; after the meal, Jorge Valdano told us about the first time he faced Boca as a visiting player. Lacing up his boots in the locker room, he said, everything around him seemed to be moving. One of the veterans came over to reassure him: "It's not you, pibe, it's the stadium." Playing at La Bombonera means overcoming an arena about to collapse under the weight of its own passion. No other ground imposes itself so forcefully upon its visitors.

In his stupendous book Boquita , Martín Caparrós reminds us that it was Argentina where fans were first baptized as "the twelfth man." Accustomed to adversity, we Mexicans consider the scoreline a suggestion we can ignore. Argentine fans, on the other hand, seek to alter the result using three basic tactics: holding their breath, insulting their rivals, and singing love songs. It's no accident that one of the most prominent barras bravas is called La Doce, or "the twelfth." These ultras aren't there to merely watch a game, but to participate in it through their shouts.

For years, magical realism has gone missing from Latin American literature, finding refuge in aviation instead. To fly across the continent is to endure a saga of detours, delays, and strange schedules that lead to a parallel reality. Maybe air traffic control is cheaper in the early morning and this determines the itineraries of the hemisphere. At any rate, I found myself on a four-hour predawn flight from Bogotá to Buenos Aires. Anyone without the meditative powers of a yogi arrived at the destination a zombie. Among the many surprises to come that day, I was made to experience May 4 in an altered state of time.

The relationship between football and aviation is no trifle—the Copa Libertadores will never be truly competitive until the continent modifies its fixture calendar and flight paths. When I was a kid, I used to take medications with labels that read "Shake well before using." Thanks to a miserable night on the plane, I arrived at the Superclásico in a state of total agitation.

Entering the stadium was likewise an extreme sport. I was lucky to be accompanied by my friend Leo Tarifeño, a diehard River fan who had sworn to never set foot in La Bombonera.

Leo is convinced that Argentines live for confrontation, eagerly disregard established rules, object impulsively, and justify themselves only through negativity, taking issue with anything they don't agree with. After asserting this theory, he put it into practice. When I praised the chanting of the Boca fans, he replied, "Deep down, their joy is bitter."

Being with Leo was the opposite of having a human shield. The route to the stadium was blocked, so we made our way through a wasteland thick with the invigorating smoke from street stalls selling choripán. Our barren surroundings gradually became a funnel. Police barricades flanked us on either side. We continued on until someone—an invisible leader far ahead—committed a grave mistake. The crowd was pushed back by the sound of riot guns and retreated to a checkpoint, where Leo and I asked for directions to the press box. An officer waved his hand as if trying to hypnotize us. We "understood" to make our way to the other side of a roundabout, but instead we ducked into the first alley we came across. Another crowd pulled us in before being repelled again by the riot guns. Everyone ran en masse to

Week 24: vroom vroom

Monica Dinculescu

Weird flex: been making cheese fondue for lunch because I make the rules here.

One of the (many) amazing things about being a Canadian living in California is that this state (unlike, say, no-rules-nevada) doesn't recognize non-US driving licenses. This means that after almost 20 years of getting my first driver's license I had to do the whole circus show (written test, road test, being treated like a teenager, etc) to be able to drive here. This week I finally bit the bullet and a) drove daily which is wild and b) did my road test (passed with flying colours, I know you're wondering). Side effect: I don't have to use my Quebec license when I get IDed in bars and go through through the whole apology that my birthday is in fact in french. Bouncers love that.

Speaking of, I've misplaced my Quebec licence and that's very irritating.

Euro 2020 is on! As per, my heart team is the Netherlands and they're doing surprisingly well! They've been #blessed with a pretty easy group (rip group F) and I am not looking that gift horse in the mouth.

It's a month that ends in y so I'm trying to redo my site. Inspos: I really like how clean Karolina's is; I'm low key of obsessed with this absurd cursor from CDL.

There's a bear in Tahoe that I'm 100% convinced is just trolling our dog. He spent a whole day casually pacing up and down next to our house, which sends Hopper into a fit of barking hysteria.

Books: I finished Midnight Library and really didn't like it, and Convenience Store Woman and it reminded me a lot of absurd theatre plays. I'm starting Klara and the Sun now.

Book Riot's Deals of the Day for March 29, 2026

Deals · Book Riot

Today's Featured Book Deals

$1.99

$1.99

$1.99

$2.99

$2.99

$2.99

$1.99

$1.99

In Case You Missed Yesterday's Most Popular Book Deals

$1.99

$2.99

$4.99

$0.99

The Seventh Bride by T. Kingfisher Get This Deal

Side effect: I don’t have to use my Quebec license when I get IDed in bars and go through through the whole apology that my birthday is in fact in french.

Monica Dinculescu

Week 14

Monica Dinculescu

Let's not bury the lede: I quit my job. After 8 years of working there, Google feels like a very different company than the one I joined, one that aligns less and less with my values, and it was time to move on. It sucks, because my immediate team was a group of wonderful people who do great work; I will miss them, and the work I did, greatly. I haven't had a vacation longer than 2 weeks since I was 18-ish, because I worked every summer and was too much of a naive workaholic to think I should take more than 1 week between jobs, so I am giving myself 6 months before I start something new this time around. It's stressful to not have a plan, and I am bad at relaxing, but I have full confidence I can learn to overachieve at this as well.

Didn't I tell y'all I was worried SOMETHING was going to happen and I wasn't going to get my vaccine? Here I am, 3 hours before my J+J stabbie stabby, reading about how my appointment is cancelled because it has an incredibly rare clotting side effect. One that is 10000 more rare than the usual clots women get threatened with for taking birth control and that absolutely nobody blinks an eye at anymore. This feels a lot like fabricated concern; we already don't care about women getting blood clots. 1 in over 6 million? Fam, that's better odds than getting in a car or not getting e-coli from Chipotle. Stop this.

I've become very bad at telling stories, which is 100% a result of the panini and not being forced to socialize with people. I've noticed several times in recent weeks where I'm telling a story and can see the life leaving from people's eyes. I am bored of what I am saying as I am saying it. I am not ready to move to the midwest yet; I must fix this.

This is a very good tweet. Abolish the police already. Stop giving guns to insecure people who think a uniform gives them the right to kill others because of the colour of their skin. Stop giving guns to incompetent people who apparently aren't trained enough to tell the difference between a gun and a taser, but are confident they have a right to use either. Stop giving guns to people.

I am working on 2 linocuts. I should probably start working on more generative stuff, since JavaScript is my bread and butter and should somehow prove to my future employers I still got it. Or something. I am a deflated balloon after finding out my vaccine was cancelled and writing about yet another dead black man above so I don't really know how to end this update on a good note.

ICYMI: The Week's Biggest Book News

Rebecca Joines Schinsky · Book Riot

Welcome to Today in Books, our daily round-up of literary headlines at the intersection of politics, culture, media, and more.

Here are the stories we covered ourselves on Book Riot this week.

"I Will Not Comply." : The Librarian Fighting for the Freedom to Read

The Most Read Books on Goodreads This Week

Manipulating the Law: Dismantling the Miller Test and Exploiting the "Government Speech" Doctrine

The Bestselling Books of the Week, According to All the Lists

And for All Access members, here are all the interesting links we bookmarked that didn't make the cut for full Today in Books coverage.

This content is for members only. Visit the site and log in/register to read.

monica.css

Monica Dinculescu

Back in the day when I worked on Polymer I got used to relying on a bunch of useful CSS classes that at the time we called iron-flex-layout . They were there partly because flexbox was a sadness on IE and you needed to say everything 3 times to maybe get it right twice, and add some very special flex-basis: 0.000000001px “bug fixes” that tbh nobody should ever have to write by hand. But they were also there because it’s kind of nice to say and for it to just work.

Some years later, it’s now 2020, and flexbox is really good everywhere! We don’t need iron-flex-layout anymore, but tbh I still want to say and for it to just work.

I know there are tons of CSS frameworks out there like tachyons that can do this for me, but most of these frameworks do too much for me. I don’t work on large projects that need design systems, and I don’t want every possible padding and margin and colour and flexbox configuration in the world. I just want the ones that I know I end up using in every project. So here is monica.css : my very own CSS framework, which I copy paste at the beginning of every CSS file and take it from there. It’s already minified and bundled (because you copy pasted it) so dare I say: fast loading and efficient? 🙃

```
* { box-sizing : border-box } [ hidden ] { display : none ! important } [ disabled ] { pointer-events : none ; opacity : 0.3 } .horizontal { display : flex ; flex-direction : row ; justify-content : space-between } .vertical { display : flex ; flex-direction : column } .center { justify-content : center ; align-items : center } .flex { flex : 1 } html { –spacing-xs : 8px ; –spacing : 24px ; –spacing-s : 12px ; –spacing-m : 36px ; }
```

Cat-DNS: learning about DNS with cats

Monica Dinculescu

I talked about Cat-DNS at Cascadia.js , and it wasn’t terrible! There is a video. Of me talking! On the internet! What a future we live in.

=^..^=

The internet needs more cats. DNS servers are the authority on all things internet. Therefore, the best DNS server is the one that resolves everything to cats. This talk is about that.

We’re going to walk through the basics and find out how DNS servers work, how you can talk to a DNS server if you’re a browser, and how to talk back to a browser if you are a DNS server. I’ll show you how you can write your own DNS server in less than 200 lines of JavaScript, but perhaps most importantly, why you probably shouldn’t.

And have I mentioned the cats? There are definitely cats.

BRIEFS

IN BRIEF

New Scientist, New Scientist: There are a few simple things you can do to make your digital life much more secure, says cybersecurity expert Jake Moore - follow these tips to tighten up your passwords

New Scientist, New Scientist: Do we all see the same red? Or feel joy and sadness alike? Mapping how our inner experiences relate to one another could finally reveal how physical processes in the brain give rise to consciousness

New Scientist, New Scientist: The neurodegenerative condition chronic traumatic encephalopathy appears to be driven by damage to the blood-brain barrier due to repetitive head injuries, like those that occur in boxing. This suggests that drugs that strengthen this barrier could prevent or slow the condition

New Scientist, New Scientist: In a randomised trial, men who experience premature ejaculation benefitted from using an app to learn techniques for extending intercourse

New Scientist, New Scientist: Strong magnets tend to be large and power-hungry, but a new design has produced a powerful magnet that fits in the palm of your hand, making it more practical and affordable

New Scientist, New Scientist: A growing body of research shows that we tend to discount a person's good deeds if they stand to benefit from them. Columnist David Robson explores where this instinct comes from – and whether we should resist it

New Scientist, New Scientist: We shouldn't dismiss flowers as merely ornamental – these blooms are world-changers, argues a vivid new book by David George Haskell. Michael Marshall is mostly convinced

Travis Turner, Evil Martians Chronicles: Authors: Victoria Melnikova, Head of New Business, Host of Dev Propulsion Labs, and Travis Turner, Tech Editor

Topics: Case Study, Developer Community

Evil Martians is a developer tools consultancy. Founded in 2006, we work with about 40 early-stage startups a year, mostly seed to Series B. We helped bolt.new scale from zero to $40M+ ARR in 5 months. Teleport and Tines reached unicorn status with Martians on their teams.

Established nearly 20 years ago, Evil Martians is a trusted software development consultancy for developer tools startups.

Read more

New Scientist, New Scientist: After the passing of physicist Anthony Leggett, columnist Karmela Padavic-Callaghan remembers their personal connection with this giant of quantum physics, and explores the legacy of his enduring recipe for testing the edges of the quantum world

New Scientist, New Scientist: A pig's brain has been frozen with its cellular activity locked in place and minimal damage. Some believe the same could be done with the brains of people with a terminal illness, so their mind can be reconstructed and they can "continue with their life"

Archivist, Medieval Archives: By the mid-1020s, Cnut the Great was arguably the most powerful man in Northern Europe. As the king of England and Denmark, he was no longer just a Viking warlord but the ruler of the North Sea Empire. Cnut was rebuilding the empire first forged by his father, Sweyn Forkbeard, reclaiming it piece by piece. His influence stretched from the Irish Sea to the Baltic. The King of Sweden…

Source

New Scientist, New Scientist: We've always thought that Tyrannosaurus rex was an unchallenged apex predator during the dying days of the dinosaurs. But a fresh look at controversial fossils has prompted palaeontology's biggest-ever U-turn

The Independent Courier

Vol. 1, No. 018 · 2026-03-30

Curated and composed automatically.